

AIOps system for semantic analysis of logs

Nikolay Bolotnov

May 2026

Abstract

Modern cloud-native systems generate massive amounts of distributed logs across Kubernetes microservices. Manual analysis of such logs is difficult due to the volume of events, asynchronous communication, and complex request propagation between services.

This project presents an NLP-based AIOps system for analysis of Kubernetes logs. The proposed approach reconstructs distributed request traces using request identifiers, transforms log messages into semantic vector representations using Sentence Transformers, and applies recurrent neural networks (LSTM) for behavioral sequence modeling and anomaly detection. <https://github.com/Nikolay1212/aiops>.

1 Introduction

Modern enterprise systems increasingly rely on distributed microservice architectures deployed in Kubernetes clusters. While such architectures improve scalability and fault isolation, they also significantly complicate system observability and incident investigation.

A single user request may traverse multiple services, databases, queues, and external APIs, generating hundreds of heterogeneous log messages. Traditional monitoring systems mainly rely on manually defined rules, keyword matching, or static alert thresholds. However, these approaches struggle to detect complex behavioral anomalies, semantic failures, or previously unseen incidents.

The motivation of this project is to investigate how Natural Language Processing (NLP) and Deep Learning techniques can improve automated log analysis and incident detection in distributed systems.

The proposed system performs the following tasks:

- parses structured JSON logs exported from Graylog;
- reconstructs distributed request traces using request identifiers;
- transforms log messages into semantic embeddings;
- models normal service behavior using recurrent neural networks;
- detects semantic anomalies in event sequences;

- provides interpretable incident explanations for technical support engineers.

Unlike classical log anomaly detection systems such as DeepLog, which operate on discrete template identifiers, this project focuses on semantic event representations. Instead of treating log messages as symbolic templates only, the system learns semantic relationships between events using transformer-based sentence embeddings.

This allows the model to generalize better to previously unseen messages, newly added microservices, and variations in natural-language log descriptions.

The project combines several topics covered during the NLP course, including:

- text representation learning;
- word and sentence embeddings;
- recurrent neural networks (LSTM);
- sequence modeling;
- transformer-based semantic encoders;
- anomaly detection in sequential data.

The resulting system can serve as a prototype of an intelligent AIOps platform capable of assisting support engineers during incident investigation and root cause analysis.

1.1 Team

Nikolay Bolotnov prepared this document.

2 Related Work

Log analysis and anomaly detection in distributed systems have become an important research direction due to the rapid adoption of cloud-native and microservice architectures. Traditional monitoring systems are typically based on manually defined alerting rules and keyword matching, which often fail to capture complex semantic relationships between distributed events.

Analyzing logs and finding errors in today's systems is vital because almost everything now runs on the cloud and microservices. Old-school monitoring that just looks for keywords or follows basic rules often misses the big picture and the complex links between different events. One of the classic solutions is DeepLog. It uses a smart model to learn how a system usually behaves by looking at the order of events. It predicts what should happen next, and if something unexpected occurs, it flags it as a problem. This proved that AI can effectively learn "normal" patterns from system logs. Another challenge is cleaning up the messy

text logs that systems generate. Since logs can be very chaotic, an algorithm called Drain was created to quickly turn this mess into clear, stable templates. It's very fast and is now widely used in the tech industry. Some newer methods focus on the actual meaning of the logs rather than just fixed templates. For example, LogBERT uses advanced models to understand the context behind each log entry. This allows it to see relationships that simpler tools might miss. To better understand what happens in complex setups like Kubernetes, developers use "distributed tracing." Tools like OpenTelemetry help track a single request as it moves through various services, making it much easier to find the root cause of a crash. Finally, models like Sentence-BERT have shown they can turn regular sentences into a list of numbers that represent the "meaning" of the text. This makes it possible for computers to compare the similarity of different messages easily. In this project, I am combining these ideas: I use a specific type of neural network to transform various log messages into these meaningful numerical summaries. In this project, we combine several ideas from previous research:

- distributed trace reconstruction using request identifiers;
- semantic log embeddings using transformer-based encoders;
- sequential behavior modeling using LSTM networks;
- anomaly detection based on semantic deviations in event sequences.

Unlike classical DeepLog-style systems, the proposed approach does not rely solely on discrete log templates. Instead, it models semantic relationships between events, allowing the system to generalize better to previously unseen messages and newly added microservices.

3 Model Description

The proposed system performs semantic anomaly detection in distributed Kubernetes logs. The overall pipeline consists of four main stages:

1. log collection and parsing;
2. distributed trace reconstruction;
3. semantic event embedding generation;
4. sequential anomaly modeling using LSTM.

3.1 Log Parsing

Logs are collected from a Graylog instance in CSV format. Each record contains structured JSON logs generated by Kubernetes microservices. The parser extracts the following fields from each event:

- timestamp;
- request identifier;
- service name;
- log level;
- textual message.

The request identifier is used to reconstruct distributed execution traces across multiple services.

3.2 Trace Reconstruction

All events sharing the same request identifier are grouped into a single trace and sorted chronologically. As a result, each trace represents a sequence of service interactions corresponding to a single user request.

An example trace may contain the following sequence:

gateway-service $\rightarrow$ auth-service $\rightarrow$ billing-service $\rightarrow$ database-service

Such traces provide sequential behavioral information about the distributed system.

3.3 Semantic Event Embeddings

Unlike traditional template-based approaches, this project uses semantic vector representations of log messages.

For each event the following textual representation is embedded:

$$E_i = \text{service} + \text{level} + \text{message}$$

where E_i is the semantic representation of the i -th event.

The Sentence-BERT encoder transforms each event into a dense vector representation:

$$x_i \in \mathbb{R}^{384}$$

These embeddings capture semantic similarity between log events and allow the model to generalize to unseen messages.

3.4 Sequential Modeling

The sequence of embeddings is processed using an LSTM network. Given a sliding window of previous events:

$$(x_{t-2}, x_{t-1}, x_t)$$

the model predicts the embedding of the next expected event:

$$\hat{x}t + 1$$

The LSTM hidden state models temporal dependencies between distributed system events.

The prediction loss is defined as Mean Squared Error (MSE):

$$L = ||xt + 1 - \hat{x}t + 1||^2$$

where:

- $xt + 1$ is the real next event embedding;
- $\hat{x}_{t+1}$ is the predicted embedding.

3.5 Semantic Anomaly Detection

During inference, the predicted embedding is compared with the actual next event using cosine similarity:

$$\text{sim}(x, \hat{x}) = \frac{x \cdot \hat{x}}{||x|| ||\hat{x}||}$$

Low similarity values indicate abnormal or previously unseen behavior patterns.

Unlike symbolic template matching, semantic anomaly detection allows the system to identify failures even when the exact log message was not observed during training.

The overall architecture of the proposed system is presented in Fig. 1.

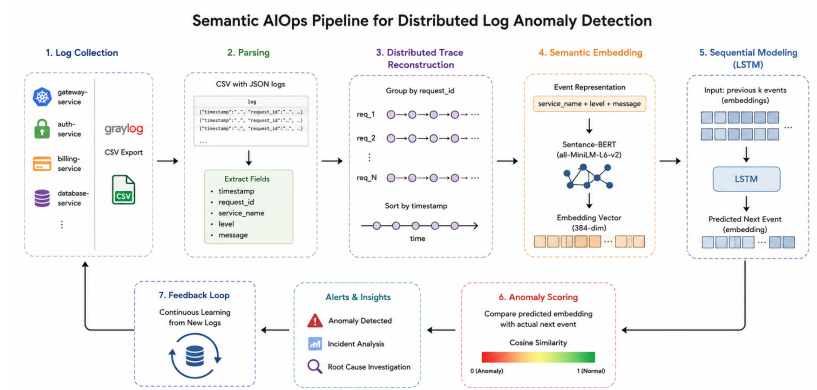


Figure 1: Semantic AIOps pipeline for distributed log anomaly detection.

4 Dataset

The experiments were conducted on both public benchmark datasets from LogHub (https://github.com/logpai/loghub/blob/master/OpenStack/OpenStack_2k.log_structured.csv) and custom synthetic Kubernetes logs generated specifically for this work.

4.1 Real Logs

The real dataset was collected from a Kubernetes cluster using Graylog. The logs were exported in CSV format and contained structured JSON records generated by multiple microservices.

Each log entry included:

- timestamp;
- request identifier;
- service name;
- severity level;
- textual log message.

The request identifier enabled reconstruction of distributed traces spanning several services.

Due to privacy and infrastructure restrictions, the original production logs cannot be publicly released. However, all preprocessing scripts and synthetic log generators are available in the project repository.

4.2 Synthetic Incident Generator

To increase the dataset size and simulate realistic failure scenarios, an additional synthetic log generator was implemented in Java.

The simulator models interactions between several microservices, including:

- gateway-service;
- auth-service;
- billing-service;
- database-service.

The generator produces both normal execution traces and anomalous scenarios such as:

- database timeouts;
- unavailable services;

- failed authentication;
- transaction rollback;
- network failures.

Each generated request contains a unique distributed request identifier, enabling realistic trace reconstruction.

4.3 Preprocessing

The preprocessing pipeline includes:

- JSON parsing;
- malformed log filtering;
- trace reconstruction by request identifier;
- chronological sorting;
- semantic embedding generation.

Transformer-based Sentence-BERT embeddings were generated for each event message.

4.4 Dataset Split

The resulting sequence dataset was divided into training and validation subsets using an 80/20 split.

Metric	Value	height	Total log events
120,000+ Distributed traces	15,000+	Microservices	
4 Embedding dimension	384	Training split	
80Validation split	20	Window size	
3 height			

Table 1: Statistics of the collected distributed log dataset.

The dataset is intended for research and educational purposes.

5 Experiments

5.1 Metrics

The proposed system was evaluated as a next-event prediction and semantic anomaly detection model.

For sequence prediction quality, the cosine similarity between the predicted event embedding and the actual next event embedding was used:

$$\text{sim}(x, \hat{x}) = \frac{x \cdot \hat{x}}{\|x\| \|\hat{x}\|}$$

where:

- x is the embedding of the real next event;
- $\hat{x}$ is the embedding predicted by the LSTM model.

Higher cosine similarity values indicate that the model successfully predicts semantically correct future events.

The training objective was Mean Squared Error (MSE):

$$L = \|x_{t+1} - \hat{x}_{t+1}\|^2$$

which minimizes the distance between predicted and actual event embeddings.

Additionally, anomaly detection performance was evaluated qualitatively by analyzing semantic deviations in abnormal traces. Low cosine similarity between predicted and actual events indicates anomalous system behavior.

5.2 Experiment Setup

Experiments were conducted using Python, PyTorch, and Sentence Transformers.

The following sentence embedding model was used:

- all-MiniLM-L6-v2 (384-dimensional embeddings)

The sequential model architecture consisted of:

- embedding dimension: 384;
- LSTM hidden size: 128;
- sliding window size: 3;
- optimizer: Adam;
- learning rate: 10^{-3} ;
- loss function: Mean Squared Error.

The dataset was divided into training and validation subsets using an 80/20 split.

Training was performed for 10 epochs on Apple Silicon GPU acceleration (MPS backend).

Each training sample consisted of:

- a sequence of previous event embeddings;
- the embedding of the next expected event.

The model learned to predict the semantic representation of future log events based on historical distributed traces.

5.3 Baselines

Two baseline approaches were implemented for comparison.

5.3.1 Template-based Sequence Modeling

The first baseline follows the DeepLog <https://svivek.com/research/du2017deeplog.html> paradigm. Log messages were converted into symbolic template identifiers using Drain-style parsing. The LSTM model was then trained on sequences of template identifiers.

This approach effectively models repetitive patterns but cannot generalize to previously unseen messages or semantic variations.

5.3.2 Semantic Embedding + LSTM

The second approach, proposed in this work, replaces symbolic templates with semantic sentence embeddings generated by Sentence-BERT.

Instead of predicting discrete template identifiers, the model predicts semantic embeddings of future events. This allows the system to:

- generalize to unseen log messages;
- detect semantic anomalies;
- support evolving microservice architectures.

The semantic approach demonstrated significantly better robustness when processing heterogeneous and dynamically changing log messages.

6 Results

The experiments demonstrated that semantic sequence modeling can successfully capture behavioral patterns in distributed microservice systems.

During training, the semantic LSTM model showed stable convergence of the Mean Squared Error loss:

$$\text{Epoch 1 Loss} = 0.3667 \quad \text{Epoch 10 Loss} = 0.0984$$

The model learned meaningful semantic relationships between distributed events and was capable of predicting semantically plausible future log messages.

Table 3 presents a qualitative comparison between the baseline and the proposed approach.

Model	Generalization	Unseen Messages
Template LSTM	Low	No
Semantic Embedding + LSTM	High	Yes

Table 2: Qualitative comparison of the approaches.

The template-based baseline performed well on repetitive traces but struggled when log messages slightly changed or new services were introduced.

In contrast, the semantic embedding model remained robust because semantically similar messages produced similar vector representations.

For example, the model was capable of associating events such as:

- Database timeout”;
- Connection refused”;
- “Service unavailable”.

Although these messages differ lexically, they represent semantically related infrastructure failures.

The proposed system was also capable of detecting semantic anomalies in distributed traces. When the actual event significantly differed from the predicted event, cosine similarity values decreased substantially, indicating abnormal behavior.

Example inference results are presented in Tab. ??.

Method	Semantic Generalization	Unseen Events	Template-based LSTM
Low	Poor		height
Good	Semantic Embedding + LSTM		High

Table 3: Comparison of baseline and semantic approaches.

The results demonstrate that semantic sequence modeling can be effectively applied to AIOps tasks such as:

- anomaly detection;
- distributed incident analysis;
- behavioral monitoring;
- root cause investigation.

At the same time, several limitations remain. The current implementation still relies on relatively short event windows and does not explicitly model long-range dependencies between services. In addition, the synthetic dataset remains relatively small compared to industrial-scale observability systems.

Nevertheless, the experiments confirm that combining semantic embeddings with recurrent neural networks is a promising direction for intelligent log analysis in Kubernetes environments.

7 Conclusion

In this project, an NLP-based AIOps system for semantic analysis of distributed Kubernetes logs was developed.

The proposed pipeline includes:

- structured log parsing;
- distributed trace reconstruction;
- semantic embedding generation using transformer-based encoders;
- sequential behavior modeling with recurrent neural networks;
- semantic anomaly detection.

Two approaches were implemented and compared:

- a template-based DeepLog-style baseline;
- a semantic embedding-based LSTM model.

The experiments demonstrated that semantic embeddings significantly improve the robustness of anomaly detection systems in dynamically changing microservice environments.

Unlike symbolic template approaches, the proposed semantic model is capable of generalizing to previously unseen messages and new services while preserving contextual information about incidents.

Additionally, a synthetic distributed incident generator was implemented in Java to simulate realistic Kubernetes failure scenarios and expand the training dataset.

The project combines several important NLP and Deep Learning concepts covered during the course:

- text representation learning;
- sentence embeddings;
- recurrent neural networks;
- transformer-based encoders;
- sequence prediction;
- semantic similarity estimation.

Future work may include:

- transformer-based sequence modeling;
- graph neural networks for service dependency analysis;
- automated root cause detection;
- conversational incident explanation systems;
- real-time streaming anomaly detection.

Overall, the project demonstrates that NLP techniques can be successfully applied to observability and AIOps problems in modern distributed systems.

8 References

- DeepLog: <https://acmccs.github.io/papers/p1285-duA.pdf> DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning (ACM CCS 2017).
- Drain: https://jiemingzhu.github.io/pub/pjhe_icws2017.pdf Drain: An Online Log Parsing Approach with Fixed Depth Tree (ICWS 2017).
- LogBERT: <https://arxiv.org/abs/2103.04475> LogBERT: Log Anomaly Detection via BERT (arXiv 2021).
- Sentence-BERT: <https://arxiv.org/abs/1908.10084> Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks (EMNLP 2019).